CLI

AI Pro CLI

AI Pro CLI는 오픈소스 AI 에이전트인 gemini-cli를 기반으로 개발한 **CLI 개발 도구**입니다.
터미널을 통해 간편한 액세스를 제공하여 프롬프트에서 모델을 바로 연결하는 사용성을 제공합니다.

주요 특징

- 다양한 cli 툴을 활용한 소스코드 검색 및 수정
- 사용자가 다양하게 설정할 수 있는 Prompt
- MCP Server에 연결하여 컨텍스트 확장
- Chat History 압축을 통한 컨텍스트 유지

주요 슬래시 커맨드 일람

Command	Function
`/help`	슬래시 커맨드의 목록을 표시한다.
`!`	Shell 명령어 실행. Windows 의 경우 cmd 에서 사용 가능한 명령어가 사용 가능하다.
`/init`	프로젝트 구조를 읽어 AGENT.md 를 생성한다.(2번항목 참고)
`/clear`	현재 chat history 및 화면 초기화
`/docs`	Gemini CLI의 docs로 이동
`/quit`	CLI를 종료
`/mcp`	등록된 mcp 툴 목록을 조회

Command	Function
`/memory`	메모리 관련 작업
`/model`	사용 가능한 모델을 확인하고, 선택 가능

※ AI Pro CLI는 gemini cli를 기반으로 하여 서비스를 제공하고 있습니다.
이에 따라 설정 및 사용방법에 대해서는 AI Pro CLI의 버전정보를 확인하시어, Gemini CLI 공식문서와 가이드 링크를 참고해주시기 바랍니다. 자세한 내용과 최신 정보를 확인하시려면 아래 문서를 확인해 주세요.
사용가이드 바로가기

AGENTS.md 활용

AGENTS.md란?

- `AGENTS.md`는 프로젝트 내에서 코드 에이전트(Code Agent)를 비롯한 에이전트들이 어떻게 동작하고, 어떻게 구성·연결되는지 정의한 문서입니다. 이 문서는 단순 개념 설명을 넘어서, **실제로 코드 에이전트가 작동할 때 참고하는 설계 지침 역할**을 합니다.
- `AGENTS.md`가 잘 작성되어 있어야 AGENT는 명확하게 정의된 절차와 규칙을 기반으로 동작해 일관된 결과를 제공합니다.
- 때문에 AI Pro CLI와 같은 AGENT 도구를 이용해 작업을 수행할 경우 **`AGENTS.md` 및 규칙에 대한 문서 정의가 필수적**입니다.

생성방법

아래 명령어를 입력하여 프로젝트 폴더 내에 `AGENTS.md`파일을 프로젝트 루트에 자동으로 생성해줍니다.

```
/init
```

```
> AIPRO

Tips for getting started:
1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. Create AGENTS.md files to customize your interactions with AIPRO.
4. /help for more information.

> /init

ℹ Empty AGENTS.md created. Now analyzing the project to populate it.

✦ I'll analyze the current directory structure and create a comprehensive AGENTS.md
  file.

  First, let me explore the project structure by reading key files:

  ✓ ReadFile README.md

✦ Let me check the main.py file to understand the project better:

  ✓ ReadFile main.py

✦ Let me check the configuration files to understand the project settings:

  ✓ ReadFile config\settings.py

✦ Let me check the route configuration to understand the API endpoints:

✦ Based on my analysis of the codebase, I'll now create a comprehensive AGENTS.md file
  that documents this project:

  ? WriteFile Writing to AGENTS.md ←

  ... first 102 lines hidden ...
  101
  102 ### Database Models
  103
  104 Models are defined using SQLAlchemy ORM with:
  105 - Audit fields (created_at, updated_at) inherited from `AuditTable
  106 - Repository pattern for data access operations
  107 - Enum types for categorical values
  108
  109 ### Authentication
  110
  111 The system implements two authentication methods:
  112 1. Service-based authentication with `X-Service-Id` header and
      `Authorization: Bearer <token>`
  113 2. User key authentication for Open API endpoints with
```

```
`Authorization: Bearer <user_key>`

### Error Handling

- Custom exception handlers in `common/error/custom_exception_handler.py`
- Middleware for catching exceptions globally
- Standardized error responses with HTTP status codes

### Logging

- Structured logging with correlation IDs
- Configurable log levels and output formats
- Specialized filters for health check requests

### Configuration

Environment-specific configuration is managed through:
- Pydantic Settings with `.env` file support
- Separate configuration files for different environments
- Clear separation of application, server, database, and API settings

## Testing

Currently, there are no explicit test files in the project structure. Tests should be added following the project's architectural patterns:
- Unit tests for services and utility functions
- Integration tests for endpoints
- Mock external dependencies (LLM services) in tests

## Deployment Considerations

- Configure production database credentials in `.env`
- Set appropriate log levels for production
- Adjust worker count and auto-reload settings
- Ensure Redis is properly configured for caching
- Set up proper authentication for service accounts

Apply this change?

● 1. Yes, allow once
  2. Yes, allow always
  3. Modify with external editor
  4. No, suggest changes (esc)

Press ctrl-s to show more lines
⠼ Waiting for user confirmation...
```

AGENTS.md 생성

`AGENTS.md`에 아래 정보들을 포함해 코드에이전트의 프로젝트 이해를 도와 답변 품질을 높일 수 있습니다.

- Project Overview
  - Key Technologies

    - Core Features
- Building and Running
    - Prerequisites
    - Build Process
    - Running the Application
- Development Conventions
    - Code Structure
    - Testing Guidelines
    - Database Migrations
    - API Development Standards
    - Pull Request Process

#작성예시

```
# Project Overview
- This is a React + TypeScript project with multiple packages in the /packages folder.
- The main service handles user authentication and API interactions.

## Dev Environment
- Run `pnpm install` to install dependencies.
- Use `pnpm run dev` to start the development server.

## Testing
- Run `pnpm test` to execute all tests.
- Fix failing tests before submitting PRs.

## PR Guidelines
- Title format: [package-name] brief description
- Run lint and tests before committing.
```

환경 설정

- 각종 환경 설정은 ~/.gemini 폴더의 `settings.json` 파일 내에 있습니다.

```
# Sample of `settings.json`
{
  "security": {
  "auth": {
    "selectedType": "aipro"
  }
},
  "tools": {
    "useRipgrep": true
  },
  "aipro": {
    "apiKey": "your api key",
    "model": "glm-4.7",
    "baseUrl": "https://aipro.sdsdev.co.kr/open/api/v1"
  }
}
```

- `.gemini` 폴더 내부에 아래와 같은 파일들을 둘 수 있습니다.
  - `settings.json`: 개별 프로젝트의 AI Pro CLI 설정을 관리
  - `AGENTS.md`: AI Pro에게 제공할 컨텍스트(설명, 스타일, 행동 지침 등)

> **팁**
> 프로젝트나 작업 폴더별로 `.gemini` 폴더를 생성하여 각 폴더마다 설정을 다르게 활용할 수 있습니다.

`.geminiignore` 파일 활용

- `.geminiignore`파일은 특정 파일이나 폴더를 AI Pro CLI가 처리, 분석 또는 액세스 하지않도록 제외할 수 있습니다.
  - 프로젝트 루트에 위치시켜 작업시 무시할 파일이나 폴더를 지정합니다.

```
#작성예시

node_modules/
*.log
secrets.json
.env
```

> **팁**
> 한글이 깨지는 경우, 터미널 인코딩 설정을 확인해주세요.